# Chapter 3 - Advanced Mobile Calculator

## Objective

In this exercise, you will enhance your calculator application with advanced mathematical functions, improved UI/UX, and professional features. This project will help you understand:

- Advanced state management patterns in Flutter
- Complex mathematical operations and expression parsing
- Custom animations and transitions
- Theme switching and dark mode implementation
- History management and data persistence
- Advanced gesture handling
- Unit testing for mathematical operations

## Responsibilities

As a student, you are expected to:

1. **Enhance the UI**: Implement advanced design features including animations, themes, and responsive layouts
2. **Add Scientific Functions**: Implement trigonometric, logarithmic, and other advanced operations
3. **Implement Expression Parsing**: Support complex expressions with proper operator precedence
4. **Add Calculation History**: Store and display previous calculations with persistence
5. **Create Multiple Modes**: Switch between basic, scientific, and programmer modes
6. **Implement Advanced Features**: Memory functions (M+, M-, MR, MC), parentheses support

7. **Add Settings**: Theme selection, decimal precision, angle mode (degrees/radians)
8. **Write Unit Tests**: Create comprehensive tests for calculation logic
9. **Optimize Performance**: Handle complex calculations efficiently

## Resources

### Figma Design

- **Design Link**: N/A
- **Design Specifications**:
  - **Colors**:
    - Light Theme: Primary (#1E1E1E), Secondary (#424242), Accent (#FF6B6B)
    - Dark Theme: Primary (#121212), Secondary (#2C2C2C), Accent (#4ECDC4)
  - **Font**: Roboto (Regular 16px for buttons, Medium 32px for display, Light 18px for history)
  - **Button spacing**: 12px
  - **Border radius**: 16px for buttons, 24px for display area
  - **Screen padding**: 24px
  - **Animation duration**: 200ms for button press, 300ms for mode switch

### Additional Resources

- **Math Expressions Package**: https://pub.dev/packages/math_expressions
- **Shared Preferences**: https://pub.dev/packages/shared_preferences
- **Provider State Management**: https://pub.dev/packages/provider
- **Flutter Animations**: https://flutter.dev/docs/development/ui/animations
- **Unit Testing**: https://flutter.dev/docs/cookbook/testing/unit/introduction

## Instructions

### Step 1: Project Setup

1. **Update your existing calculator project or create new**:

   ```
   flutter create advanced_calculator

   cd advanced_calculator
   ```

2. **Add required dependencies to pubspec.yaml**:

   ```
   dependencies:

     flutter:

       sdk: flutter

     provider: ^6.1.1

     shared_preferences: ^2.2.2

     math_expressions: ^2.4.0

     intl: ^0.18.1


   dev_dependencies:

     flutter_test:

       sdk: flutter

     mockito: ^5.4.4
   ```

3. **Run**: `flutter pub get`

### Step 2: Project Structure

Organize your project with proper architecture:

lib/

```
main.dart
models/
    calculation_history.dart
    calculator_mode.dart
    calculator_settings.dart
providers/
    calculator_provider.dart
    theme_provider.dart
    history_provider.dart
screens/
    calculator_screen.dart
    history_screen.dart
    settings_screen.dart
widgets/
    display_area.dart
    button_grid.dart
    calculator_button.dart
    mode_selector.dart
utils/
    calculator_logic.dart
    expression_parser.dart
    constants.dart
services/
storage_service.dart
```

**Step 3: Advanced UI Implementation**

**Display Area Features:**

1. **Multi-line Display**:
    - Current expression (scrollable)
    - Previous result (dimmed)
    - Error messages with animation
2. **History Preview**:
    - Swipeable panel showing last 3 calculations
    - Tap to reuse previous calculation
3. **Mode Indicator**:
    - Visual indicator for DEG/RAD mode
    - Memory indicator when values stored

**Button Layout:**

**Basic Mode** (4×5 grid):

| | | | |
|---|---|---|---|
| C | CE | % | ÷ |
| 7 | 8 | 9 | × |
| 4 | 5 | 6 | - |
| 1 | 2 | 3 | + |
| ± | 0 | . | = |

**Scientific Mode** (6×6 grid):

| | | | | | |
|---|---|---|---|---|---|
| $2^{nd}$ | sin | cos | tan | Ln | log |
| $x^2$ | √ | x^y | ( | ) | ÷ |
| MC | 7 | 8 | 9 | C | × |
| MR | 4 | 5 | 6 | CE | - |
| M+ | 1 | 2 | 3 | % | + |
| M- | ± | 0 | . | Π | = |

**Programmer Mode** (Features):

- Binary, Octal, Decimal, Hexadecimal conversions
- Bitwise operations (AND, OR, XOR, NOT)
- Bit shifting (<<, >>)

**Step 4: Advanced Functionality**

**State Management Structure:**

```
class CalculatorProvider extends ChangeNotifier {
  String _expression = '';
  String _result = '0';
  CalculatorMode _mode = CalculatorMode.basic;
  AngleMode _angleMode = AngleMode.degrees;
  double _memory = 0;
  bool _hasMemory = false;

  // Methods
  void addToExpression(String value);
  void calculate();
  void clear();
  void clearEntry();
  void toggleSign();
  void addPercentage();
  void addScientificFunction(String function);
  void toggleMode(CalculatorMode mode);
  void memoryAdd();
  void memorySubtract();
  void memoryRecall();
  void memoryClear();
}
```

**Advanced Features to Implement:**

1. **Expression Parser**:
    - Support for operator precedence (PEMDAS)
    - Parentheses handling
    - Implicit multiplication (2π = 2×π)
    - Error detection and handling
2. **Scientific Functions**:
    - Trigonometric: sin, cos, tan, asin, acos, atan
    - Logarithmic: ln, log, $\log_2$

- Power operations: $x^2$, $x^3$, $x^y$, $\sqrt{}$, $\sqrt[3]{}$
- Constants: π, e
- Factorial: n!

3. **Memory Functions**:

```
void memoryAdd() {
 _memory += double.parse(_result);
  _hasMemory = true;
}

void memorySubtract() {
  _memory -= double.parse(_result);
}

void memoryRecall() {
  _expression = _memory.toString();
  _result = _memory.toString();
}

void memoryClear() {
  _memory = 0;
  _hasMemory = false;
}
```

4. **History Management**:

```
class CalculationHistory {
  final String expression;
  final String result;
  final DateTime timestamp;

  CalculationHistory({
    required this.expression,
    required this.result,
    required this.timestamp,
  });

  Map<String, dynamic> toJson();
  factory CalculationHistory.fromJson(Map<String, dynamic> json);
}
```

**Step 5: Advanced Features Implementation**

**1. Theme System:**

```
class ThemeProvider extends ChangeNotifier {
  ThemeMode _themeMode = ThemeMode.system;

  ThemeData get lightTheme => ThemeData(...);
  ThemeData get darkTheme => ThemeData(...);

  void toggleTheme();
  void setTheme(ThemeMode mode);
}
```

**2. Data Persistence:**

- Save calculation history (last 50 calculations)
- Save theme preference
- Save calculator mode preference
- Save memory value
- Save angle mode (DEG/RAD)

**3. Gesture Support:**

- Swipe right on display to delete last character
- Long press on C to clear history
- Swipe up to open history
- Pinch to change font size

**4. Animations:**

- Button press animations with scale effect
- Mode transition animations
- Result display fade-in
- Error shake animation

**Step 6: Settings Screen**

Implement a settings screen with:

- **Theme Selection**: Light, Dark, System
- **Decimal Precision**: 2-10 decimal places

- **Angle Mode**: Degrees/Radians (for scientific mode)
- **Haptic Feedback**: On/Off
- **Sound Effects**: On/Off
- **History Size**: 25/50/100 calculations
- **Clear All History**: Button with confirmation dialog

**Step 7: Testing Requirements**

**Unit Tests:**

```
/// Create tests for:
test('Basic arithmetic operations', () {
  expect(calculator.calculate('5 + 3'), '8');
  expect(calculator.calculate('10 - 4'), '6');
  expect(calculator.calculate('6 × 7'), '42');
  expect(calculator.calculate('15 ÷ 3'), '5');
});

test('Order of operations', () {
  expect(calculator.calculate('2 + 3 × 4'), '14');
  expect(calculator.calculate('(2 + 3) × 4'), '20');
});

test('Scientific functions', () {
  expect(calculator.calculate('sin(30)'), '0.5');
  expect(calculator.calculate('√16'), '4');
});

test('Edge cases', () {
  expect(calculator.calculate('5 ÷ 0'), 'Error: Division by zero');
  expect(calculator.calculate('√(-4)'), 'Error: Invalid input');
});
```

**Integration Tests:**

- Button press sequences
- Mode switching
- History save/load
- Theme persistence

**Step 8: Complex Test Scenarios**

Test the following:

1. **Complex expressions**: (5 + 3) × 2 - 4 ÷ 2 = 14

2. **Scientific calculations**: sin(45°) + cos(45°) ≈ 1.414
3. **Memory operations**: 5 M+ 3 M+ MR = 8
4. **Chain calculations**: 5 + 3 = + 2 = + 1 = 11
5. **Parentheses nesting**: ((2 + 3) × (4 - 1)) ÷ 5 = 3
6. **Mixed scientific**: 2 × π × √9 ≈ 18.85
7. **Programmer mode**: 0xFF AND 0x0F = 0x0F

**Outcome of this Practice**

Upon successful completion, you will have:

- **Professional Calculator App** with advanced features matching commercial applications
- **Advanced State Management** using Provider pattern
- **Data Persistence** with SharedPreferences
- **Complex UI/UX** with animations and responsive design
- **Comprehensive Testing** with unit and integration tests
- **Clean Architecture** with proper separation of concerns
- **Advanced Problem-Solving** handling complex mathematical expressions

**Expected Features:**

- Three calculator modes (Basic, Scientific, Programmer)
- Expression evaluation with proper precedence
- Calculation history with persistence
- Dark/Light theme with smooth transitions
- Memory functions (M+, M-, MR, MC)
- Scientific functions (trig, log, power, etc.)
- Settings screen with customization
- Gesture controls
- Smooth animations
- Comprehensive error handling
- Unit tests with >80% coverage

**Way to Submit**

**Required 1: On GitHub**

1. **Create a GitHub Repository**:
    - **Name**: flutter_advanced_calculator_[your_name]
    - **README.md** must include:
        - Project description and features
        - Screenshots/GIFs of all modess
        - Architecture diagram
        - Setup instructions
        - Testing instructions
        - Known limitations
        - Future improvements
2. **Repository Structure**:

- README.md
- screenshots/
- docs/
    - ARCHITECTURE.md
    - TESTING.md
- lib/
- test/
- pubspec.yaml

3. **Documentation Requirements**:

- Code documentation for complex functions
- Architecture decision records
- API documentation for calculator logic

**Required 2: Zip Source Code File on E-Learning**

1. **Prepare Your Project**:

- Remove build/, .dart_tool/, .idea/ folders
- Include test/ folder
- Include screenshots/ folder with all modes
- Include demo video (optional, max 2 minutes)

2. **Create ZIP File**:

- **Name**: AdvancedCalculator_[StudentID]_[Name].zip

3. **Upload to E-learning**:

- Navigate to Chapter 3 submission area
- Upload ZIP file
- Add comment describing:
    - Extra features implemented
    - Challenges faced and solutions
    - Test coverage percentage

**Submission Checklist:**

- All three calculator modes implemented
- Scientific functions working correctly
- Expression parser handles complex expressions
- History persistence working
- Theme switching functional
- Settings saved and loaded
- Unit tests written with >80% coverage
- UI matches design specifications
- No compilation errors or warnings
- README complete with screenshots
- Code properly commented
- Tested on both Android and iOS (if possible)
- Performance optimized (no lag in calculations)

**Deadline**: 11:59 PM – April 24th, 2026

**Grading Criteria**

- **Advanced UI Implementation (25%)**: Theme system, animations, responsive design

- **Core Functionality (30%)**: All modes working, expression parsing, scientific functions
- **Advanced Features (20%)**: History, memory, settings, persistence

- **Code Quality & Architecture (15%)**: Clean code, proper structure, state management

- **Testing (10%)**: Unit tests, integration tests, coverage

- **Documentation (5%)**: README, code comments, architecture docs

**Bonus Points (up to 10% - Optional):**

- Landscape mode support
- Tablet/iPad optimization
- Voice input for calculations
- Graph plotting for functions
- Export history to CSV/PDF
- Custom themes creation

---

**Note**: This is an advanced project. Start early and break it down into manageable tasks. Use version control effectively with meaningful commit messages. If you encounter difficulties, consult the course forum or instructor during office hours.

**Academic Integrity**: While you may reference online resources, your implementation must be your own work. Plagiarism will result in zero credit for the assignment.